

Longest Contiguous Equal Values

We have two arrays:

```
a = [ ... ]  
b = [ ... ]
```

both of length n .

We have to merge them into one array c of length $2n$.

The important restriction is:

At every step, we can only take the first remaining element from either a or b . We want the longest contiguous part of c where all values are equal.

For example:

```
c = [1, 2, 2, 2, 5, 5]
```

The answer is:

3

For every value x , we need to know:

- What is the longest consecutive block of x in array a ?
- and: What is the longest consecutive block of x in array b ?

Suppose:

```
a = [2, 2, 2, 5, 5]
b = [1, 2, 2, 2, 2, 7]
```

For value 2:

- **a** → maximum consecutive 2 = 3
- **b** → maximum consecutive 2 = 4

When merging, we can put those blocks together:

```
2 2 2 | 2 2 2 2
```

giving:

$$3 + 4 = 7$$

So the answer for value 2 is:

7

C++ Source Code

```
#include<bits/stdc++.h>
using namespace std;
int cnta;
int cntb;
int cnt;
int main(){

    int t;
    cin>>t;
    while(t--){

        int n;
        cin>>n;

        vector<int> arr(n);
```

```

vector<int> brr(n);

for(int i=0;i<n;i++){
    cin>>arr[i];
}

for(int i=0;i<n;i++){
    cin>>brr[i];
}

unordered_map<int,int> mpa;
cnt=1;
for(int i=1;i<n;i++){
    if(arr[i]==arr[i-1]) cnt++;
    else{
        mpa[arr[i-1]]=max(mpa[arr[i-1]],cnt);
        cnt=1;
    }
}

mpa[arr[n-1]]=max(mpa[arr[n-1]],cnt);

unordered_map<int,int> mpb;
cnt=1;
for(int i=1;i<n;i++){
    if(brr[i]==brr[i-1]) cnt++;
    else{
        mpb[brr[i-1]]=max(mpb[brr[i-1]],cnt);
        cnt=1;
    }
}

mpb[brr[n-1]]=max(mpb[brr[n-1]],cnt);

int ans=0;
for(auto &it:mpa){
    int val=it.first;
    cnta=it.second;
    cntb=mpb[val];
    ans=max(ans,cnta+cntb);
}
for(auto &it:mpb){
    int val=it.first;
    cntb=it.second;
    cnta=mpa[val];
    ans=max(ans,cnta+cntb);
}

cout<<ans<<"\n";
}

```

```
    return 0;  
}
```